INFO6002 Database Management 2
Assignment 1

Task 2: Logical Database

Group Member Names:

Madhavi Latha Tripuram - C3528532

Hari Krishna Jadda - C3511138

Dhimanshi Piripun Jayasena - C3503338

Lab Timings: **Thursday 1.00pm to 3.00pm**

Course coordinator: **Prof. Mahakim Newton**

Task 2: Mapping the EER to Relational Model

Using the mapping rules, got the following relations for all entities in EER:

1) **Branch** (branchId, name, address, country, phoneNumber, currency, operationalStatus)
   Primary Key: branchId

2) **Vehicle** (vehicleId, make, model, year, registrationNumber, status, mileage, seatingCapacity, fuelType, transmissionType, baseRentalRate, branchId, categoryId)

   Primary Key: vehicleId
   Foreign Key branchId references Branch(branchId) ON UPDATE CASCADE, ON DELETE CASCADE
   Foreign Key categoryId references VehicleCategory(categoryId) ON UPDATE CASCADE, ON DELETE CASCADE

3) **VehicleCategory**(categoryId, name, description, dailyRateMultiplier)
   Primary Key: categoryId

4) **Manager_Employee**(employeeId,name,position,managerLevel, approvalAuthority, branchId)

   Primary Key: employeeId
   Foreign Key branchId references Branch(branchId) ON UPDATE CASCADE, ON DELETE CASCADE

5) **Associate_Employee**(employeeId, name, position, specialization, branchId)

   Primary Key: employeeId
   Foreign Key branchId references Branch(branchId) ON UPDATE CASCADE, ON DELETE CASCADE

6) **Customer** (customerId, fullName, contactDetails, email, licenseInformation, loyaltyTier)
   Primary Key: customerId

7) **RegularService_Service**(serviceId, name, description, status, basePrice, currency, restrictions)
   Primary Key: serviced

8) **PackageService_Service**(serviceId, name, description, status, basePrice, currency, validPeriodStart, validPeriodEnd, inclusions, exclusions, gracePeriod)
   Primary Key: serviceId

9) **PricedService**(pricedServiceId, price, pricingStartDate, pricingEndDate, approverManagerId, serviceId, branchId)

   Primary Key: pricedServiceId
   Foreign Key serviceId references Service(serviceId) ON UPDATE CASCADE, ON DELETE CASCADE
   Foreign Key branchId references Branch(branchId) ON UPDATE CASCADE, ON DELETE CASCADE
   Foreign Key approverManagerId references Employee(employeeId) ON UPDATE CASCADE, ON DELETE CASCADE

10) **ExtraService**(extraServiceId, name, description, advertisedRate)
    Primary Key: extraServiceId

11) **Booking** (bookingId, modeOfBooking, bookingDate, customerInformation, numberOfPricedServices, numberOfVehicles, serviceStartDate, serviceEndDate, paymentMethod, bookingDeposit, bookingStatus, customerId)

    Primary Key: bookingId
    Foreign Key customerId references Customer(customerId) ON UPDATE CASCADE, ON DELETE CASCADE

12) **Driver** (driverId, name, licenseInformation, contactNumbers)
Primary Key: driverId

13) **VehicleTransaction**(transactionId, transactionType, actualDateTime, vehicleCondition, mileageReading, fuelLevel, additionalCharges, bookingId, vehicleId, employeeId)

Primary Key: transactionId
Foreign Key bookingId references Booking(bookingId) ON UPDATE CASCADE, ON DELETE CASCADE
Foreign Key vehicleId references Vehicle(vehicleId) ON UPDATE CASCADE, ON DELETE CASCADE
Foreign Key employeeId references Employee(employeeId) ON UPDATE CASCADE, ON DELETE CASCADE

14) **Invoice** (invoiceId, baseRentalCharges, extraServiceCharges, totalLatePenalty, totalDamagePenalty, totalDiscount, finalAmount, paymentMethod, bookingId)

Primary Key: invoiceId
Foreign Key bookingId references Booking(bookingId) ON UPDATE CASCADE, ON DELETE CASCADE

15) **Booking_PricedService**(bookingId, pricedServiceId)

Primary Key: bookingId, pricedServiceId
Foreign Key bookingId references Booking(bookingId) ON UPDATE CASCADE, ON DELETE CASCADE
Foreign Key pricedServiceId references PricedService(pricedServiceId) ON UPDATE CASCADE, ON DELETE CASCADE

16) **Booking_Vehicle**(bookingId, vehicleId)

Primary Key: bookingId, vehicleId
Foreign Key bookingId references Booking(bookingId) ON UPDATE CASCADE, ON DELETE CASCADE
Foreign Key vehicleId references Vehicle(vehicleId) ON UPDATE CASCADE, ON DELETE CASCADE

17) **Booking_Driver**(bookingId, driverId)

Primary Key: bookingId, driverId
Foreign Key bookingId references Booking(bookingId) ON UPDATE CASCADE, ON DELETE CASCADE
Foreign Key driverId references Driver(driverId) ON UPDATE CASCADE, ON DELETE CASCADE

18) **Booking_ExtraService**(bookingId, extraServiceId, quantity)

Primary Key: bookingId, extraServiceId
Foreign Key bookingId references Booking(bookingId) ON UPDATE CASCADE, ON DELETE CASCADE
Foreign Key extraServiceId references ExtraService(extraServiceId) ON UPDATE CASCADE, ON DELETE CASCADE

Normalising the Scheme up to BCNF

According to the definitions of 1NF, 2NF, 3NF and BCNF, it is identified that relations x1, x2, ..xn are all in BCNF, since all the attributes are atomic, and there exists only one function dependency in each table, and the left side of the FD is a PK.

1. **Branch:**

   Branch(branchId, name, address, country, phoneNumber, currency, operationalStatus)

   Functional Dependencies:

   FD1: branchId → name, address, country, phoneNumber, currency, operationalStatus

   If phoneNumber is unique per branch then

   FD2: phoneNumber → branchId, name, address, country, currency, operationalStatus

   Normalization: Already in BCNF form since the only FD has a superkey as determinant

2. **Vehicle:**

Vehicle (vehicleId, make, model, year, registrationNumber, status, mileage, seatingCapacity, fuelType, transmissionType, baseRentalRate, branchId, categoryId)

Functional Dependencies:

FD1: vehicleId → make, model, year, registrationNumber, status, mileage, seatingCapacity, fuelType, transmissionType, baseRentalRate, branchId, categoryId

Assuming registration numbers are unique

FD2: registrationNumber → vehicleId (alternate Key)

Normalization: Both FDs have candidate keys as determinants vehicleId and registrationNumber. No partial or transitive dependencies. Relation is in BCNF.

3. **VehicleCategory:**

VehicleCategory(categoryId, name, description, dailyRateMultiplier)

Functional Dependencies:
FD1: categoryId → name, description, dailyRateMultiplier

if name is unique then

FD2: name → categoryId, description, dailyRateMultiplier

Normalization: Both FDs have a superkey so already in BCNF.

4. **Manager_Employee:**

Manager_Employee(employeeId, name, position, managerLevel, approvalAuthority, branchId)

Functional Dependencies:
FD1: employeeId → name, position, managerLevel, approvalAuthority, branchId

Normalization: Both FDs have a superkey , already in BCNF.

5. **Associate_Employee:**
   Associate_Employee(employeeId, name, position, specialization, branchId)

   Functional Dependencies:
   FD1: employeeId → name, position, specialization, branchId

   Normalization: Original design had redundancy with position in both superclass and subclasses. Fixed by moving position to superclass only and creating separate subclass tables. All relations now in BCNF.

6. **Customer:**
   Customer(customerId, fullName, contactDetails, email, licenseInformation, loyaltyTier)

   Functional Dependencies:
   FD1: customerId → fullName, contactDetails, email, licenseInformation, loyaltyTier

   If email is unique then,
   FD2: email → customerId, fullName, contactDetails, licenseInformation, loyaltyTier

   Normalization: Both FDs have a superkey, Therefore already in BCNF.

7. **RegularService_Service:**

   RegularService_Service(serviceId, name, description, status, basePrice, currency, restrictions)

   Functional Dependencies:
   FD1: serviceId → name, description, status, basePrice, currency, restrictions

   Normalization: Both FDs have a superkey on the left, already in BCNF.

8. **PackageService_Service:**

   PackageService_Service(serviceId, name, description, status, basePrice, currency, validPeriodStart, validPeriodEnd, inclusions, exclusions, gracePeriod)

   Functional Dependencies:
   FD1: serviceId → name, description, status, basePrice, currency, validPeriodStart, validPeriodEnd, inclusions, exclusions, gracePeriod

Normalization: Original design had repeating attributes in both subclasses. Fixed by creating superclass for common attributes. All relations in BCNF Form.

9. **PricedService:**

PricedService(pricedServiceId, price, pricingStartDate, pricingEndDate, approverManagerId, serviceId, branchId)

Functional Dependencies:
FD1: pricedServiceId → price, pricingStartDate, pricingEndDate, approverManagerId, serviceId, branchId

Normalization: Both FDs have candidate keys as determinants. Relation is in BCNF.

10. **ExtraService:**

ExtraService(extraServiceId, name, description, advertisedRate)

Functional Dependencies:

FD1: extraServiceId → name, description, advertisedRate

Normalization: Both FDs have a superkey on the left, already in BCNF Form.

11. **VehicleTransaction Entity:**

VehicleTransaction(transactionId, transactionType, actualDateTime, vehicleCondition, mileageReading, fuelLevel, additionalCharges, bookingId, vehicleId, employeeId)

Functional Dependencies:
FD1: transactionId → transactionType, actualDateTime, vehicleCondition, mileageReading, fuelLevel, additionalCharges, bookingId, vehicleId, employeeId

Assuming one transaction type per booking.
FD2: bookingId, transactionType → transactionId

Normalization: Relation is in BCNF. Additional constraints may be needed for business rules

12. Driver:

Driver(driverId, name, licenseInformation, contactNumbers)

Functional Dependencies:

FD1: driverId → name, licenseInformation, contactNumbers

Normalization: Both FDs have a superkey on the left, already in BCNF.

13. VehicleTransaction:

VehicleTransaction(transactionId, transactionType, actualDateTime, vehicleCondition, mileageReading, fuelLevel, additionalCharges, bookingId, vehicleId, employeeId)

Functional Dependencies:

FD1:transactionId → transactionType, actualDateTime, vehicleCondition, mileageReading, fuelLevel, additionalCharges, bookingId, vehicleId, employeeId

If each is unique per transaction

FD2:

(bookingId, vehicleId, transactionType) → actualDateTime, vehicleCondition, mileageReading, fuelLevel, additionalCharges, vehicleId, employeeId

Normalization: All FDs have a superkey on the left, already in BCNF.

14. Invoice:

Invoice(invoiceId, baseRentalCharges, extraServiceCharges, totalLatePenalty, totalDamagePenalty, totalDiscount, finalAmount, paymentMethod, bookingId)

Functional Dependencies:

FD1:invoiceId → baseRentalCharges, extraServiceCharges, totalLatePenalty, totalDamagePenalty, totalDiscount, finalAmount, paymentMethod, bookingId

if each booking has one invoice, then

FD2: bookingId → invoiceId, baseRentalCharges, extraServiceCharges, totalLatePenalty, totalDamagePenalty, totalDiscount, finalAmount, paymentMethod

Normalization:

All FDs have a superkey on the left, already in BCNF.

15. **Booking:**

Booking(bookingId, modeOfBooking, bookingDate, customerInformation, numberOfPricedServices, numberOfVehicles, serviceStartDate, serviceEndDate, paymentMethod, bookingDeposit, bookingStatus, customerId)

Functional Dependencies:

FD1: bookingId → modeOfBooking, bookingDate, customerInformation, numberOfPricedServices, numberOfVehicles, serviceStartDate, serviceEndDate, paymentMethod, bookingDeposit, bookingStatus, customerId

Assuming one booking per customer per day

FD2: customerId, bookingDate → bookingId, modeOfBooking, customerInformation, numberOfPricedServices, numberOfVehicles, serviceStartDate, serviceEndDate, paymentMethod, bookingDeposit, bookingStatus,

16. **Booking_PricedService:**

Booking_PricedService(bookingId, pricedServiceId)

Functional Dependencies:

FD1: bookingId, pricedServiceId → no additional attributes, just the relationship

Normalization: Already in BCNF.

17. **Booking_Vehicle:**
    Booking_Vehicle(bookingId, vehicleId)

    Functional Dependencies:
    FD1: bookingId, vehicleId →no additional attributes

    Business rule: no double-booking
    FD2: vehicleId, serviceStartDate, serviceEndDate → bookingId

    Normalization: All relations in BCNF. Additional business rules enforced through constraints

18. **Booking_Driver:**

    Booking_Driver(bookingId, driverId)

    Functional Dependencies:

    FD1: bookingId, driverId → no additional attributes

    Normalization: Already in BCNF.

19. **Booking_ExtraService:**

    Booking_ExtraService(bookingId, extraServiceId, quantity)

    Functional Dependencies:

    FD1: bookingId, extraServiceId → quantity

    Normalization: Already in BCNF.

Normalization to BCNF:

All relations are already in BCNF because:

- For each relation, the left side of every functional dependency is a superkey either the primary key or a candidate key
- There are no non-trivial functional dependencies where the determinant is not a superkey

No decomposition is needed as all relations satisfy BCNF.

Final DBDL Schema

Branch(branchId, name, address, country, phoneNumber, currency, operationalStatus)

Primary Key branchId

VehicleCategory(categoryId, name, description, dailyRateMultiplier)

Primary Key categoryId

Vehicle(vehicleId, make, model, year, registrationNumber, status, mileage, seatingCapacity, fuelType, transmissionType, baseRentalRate, branchId, categoryId)

Primary Key vehicleId

Alternate Key registrationNumber

Foreign Key branchId references Branch(branchId)

ON UPDATE CASCADE, ON DELETE RESTRICT

Foreign Key categoryId references VehicleCategory(categoryId)

ON UPDATE CASCADE, ON DELETE RESTRICT

Employee(employeeId, name, position, branchId, employeeType)

Primary Key employeeId

Foreign Key branchId references Branch(branchId)

ON UPDATE CASCADE, ON DELETE RESTRICT

ManagerDetails(employeeId, managerLevel, approvalAuthority)

Primary Key employeeId

Foreign Key employeeId references Employee(employeeId)

ON UPDATE CASCADE, ON DELETE CASCADE

AssociateDetails(employeeId, specialization)

Primary Key employeeId

Foreign Key employeeId references Employee(employeeId)

ON UPDATE CASCADE, ON DELETE CASCADE

Customer(customerId, fullName, contactDetails, email, licenseInformation, loyaltyTier)

Primary Key customerId

Service(serviceId, name, description, status, serviceType)

Primary Key serviceId

RegularServiceDetails(serviceId, basePrice, currency, restrictions)

Primary Key serviceId

Foreign Key serviceId references Service(serviceId)

ON UPDATE CASCADE, ON DELETE CASCADE

PackageServiceDetails(serviceId, basePrice, currency, validPeriodStart, validPeriodEnd, inclusions, exclusions, gracePeriod)

Primary Key serviceId

Foreign Key serviceId references Service(serviceId)

ON UPDATE CASCADE, ON DELETE CASCADE

PricedService(pricedServiceId, price, pricingStartDate, pricingEndDate, approverManagerId, serviceId, branchId)

Primary Key pricedServiceId

Foreign Key serviceId references Service(serviceId)

ON UPDATE CASCADE, ON DELETE RESTRICT

Foreign Key branchId references Branch(branchId)

ON UPDATE CASCADE, ON DELETE RESTRICT

Foreign Key approverManagerId references Employee(employeeId)

ON UPDATE CASCADE, ON DELETE RESTRICT

ExtraService(extraServiceId, name, description, advertisedRate)

Primary Key extraServiceId

Booking(bookingId, modeOfBooking, bookingDate, customerInformation, serviceStartDate, serviceEndDate, paymentMethod, bookingDeposit, bookingStatus, customerId)

Primary Key bookingId

Foreign Key customerId references Customer(customerId)

ON UPDATE CASCADE, ON DELETE RESTRICT

Driver(driverId, name, licenseInformation, contactNumbers)

Primary Key driverId

VehicleTransaction(transactionId, transactionType, actualDateTime, vehicleCondition, mileageReading, fuelLevel, additionalCharges, bookingId, vehicleId, employeeId)

Primary Key transactionId

Foreign Key bookingId references Booking(bookingId)

ON UPDATE CASCADE, ON DELETE RESTRICT

Foreign Key vehicleId references Vehicle(vehicleId)

ON UPDATE CASCADE, ON DELETE RESTRICT

Foreign Key employeeId references Employee(employeeId)

ON UPDATE CASCADE, ON DELETE RESTRICT

Invoice(invoiceId, baseRentalCharges, extraServiceCharges, totalLatePenalty, totalDamagePenalty, totalDiscount, finalAmount, paymentMethod, bookingId)

Primary Key invoiceId

Foreign Key bookingId references Booking(bookingId)

ON UPDATE CASCADE, ON DELETE CASCADE

Booking_PricedService(bookingId, pricedServiceId)

Primary Key bookingId, pricedServiceId

Foreign Key bookingId references Booking(bookingId)

ON UPDATE CASCADE, ON DELETE CASCADE

Foreign Key pricedServiceId references PricedService(pricedServiceId)

ON UPDATE CASCADE, ON DELETE CASCADE

Booking_Vehicle(bookingId, vehicleId)

Primary Key bookingId, vehicleId

Foreign Key bookingId references Booking(bookingId)

ON UPDATE CASCADE, ON DELETE CASCADE

Foreign Key vehicleId references Vehicle(vehicleId)

ON UPDATE CASCADE, ON DELETE RESTRICT

Booking_Driver(bookingId, driverId)

Primary Key bookingId, driverId

Foreign Key bookingId references Booking(bookingId)

ON UPDATE CASCADE, ON DELETE CASCADE

Foreign Key driverId references Driver(driverId)

ON UPDATE CASCADE, ON DELETE RESTRICT

Booking_ExtraService(bookingId, extraServiceId, quantity)

Primary Key bookingId, extraServiceId

Foreign Key bookingId references Booking(bookingId)

ON UPDATE CASCADE, ON DELETE CASCADE

Foreign Key extraServiceId references ExtraService(extraServiceId)

ON UPDATE CASCADE, ON DELETE CASCADE

UPDATED EER DIAGRAM

